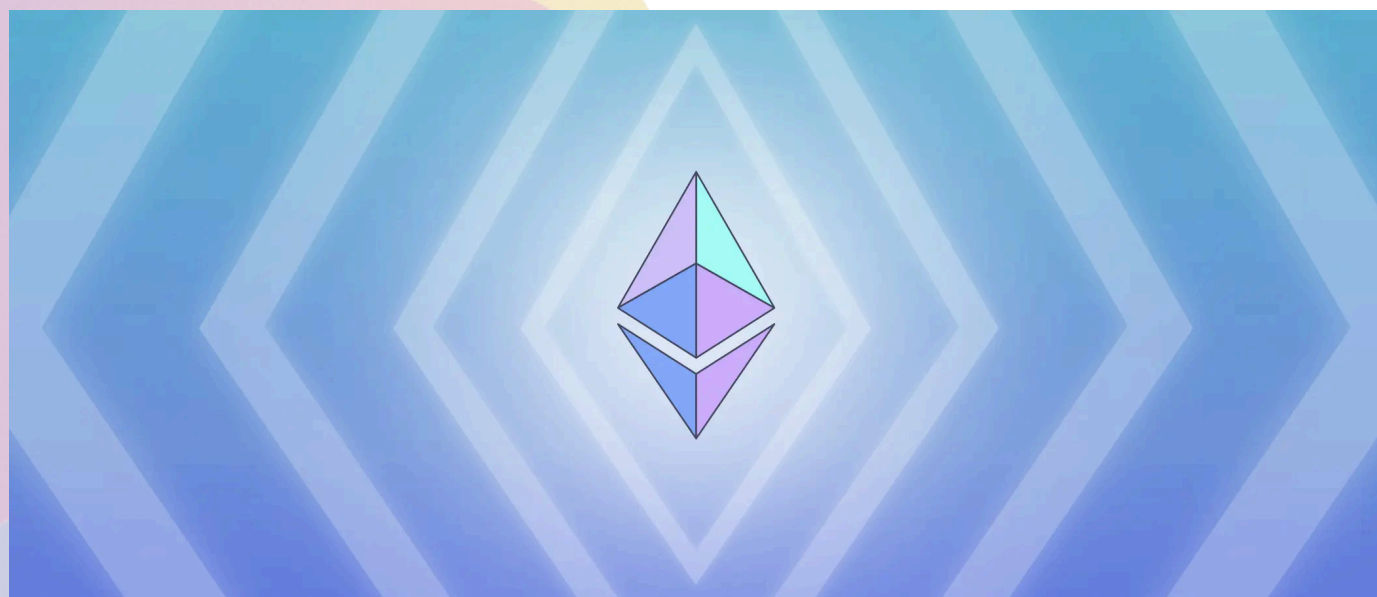


SchellingCoin: A Minimal-Trust Universal Data Feed

Posted by Vitalik Buterin on March 28, 2014

Research & Development



One of the main applications of Ethereum that people have been interested in is financial contracts and derivatives. Although financial derivatives have acquired a reputation as a highly risky and destabilizing device with the sole function of enriching speculators, the underlying concept in fact has a number of legitimate uses, some of which actually help people protect themselves against the volatility of financial markets.

The main idea here is called “hedging”, and is best explained in the context of Bitcoin, where ordinary businesses and individuals with no desire to take massive risks end up needing to deal with high volumes of a risky asset (BTC). Hedging works as follows. Suppose that Jane is a business owner who accepts Bitcoin for payments and uses it to pay employees, and on average she expects that she will need to keep 100 BTC on hand at any time. Sometimes, this amount might change; it could be 20 BTC or it could be 160 BTC. However, she is not at all excited about the prospect of seeing her BTC drop 23% in value in a single day and losing several months worth of salary. Currently, the “standard” solution is for Jane to set up her business to accept payments via BitPay or Coinbase, paying a 1% fee to have the bitcoins instantly converted into money in her bank account. When she wants to pay BTC, she would need to buy the bitcoins back and send them out, paying 1% again (if not more).

difference. In this CFD, Jane agrees to put in \$20000 worth of BTC, and get back (in BTC) \$20000 plus \$100 for every dollar that the BTC price drops. If the BTC price rises, she loses \$100 per dollar. Thus, if the value of one bitcoin decreases by \$45, Jane would lose \$4500 in the value of her bitcoins, but she would gain \$4500 in the CFD. Of course, the money does not come out of nowhere; on the other side of the contract is a speculator, betting that the price of BTC will go up, and if it does then Jane will gain in the value of BTC and lose in the CFD, and the speculator would gain in the CFD.

Given this basic ingredient, Jane has three strategies for using it to manage risk:

1. She can keep the CFD at \$100 to \$1 forever, and if her exposure is off by some amount then she can take that smaller risk.
2. Jane can have a bot constantly adjust the CFD to her supply of BTC on hand, paying some fees for this but not nearly as much as with Bitpay and Coinbase.
3. Thanks to the magic of Ethereum contracts, she can make a CFD that automatically listens to her account balance and retargets itself to her balance, forcing the speculator to assume whatever exposure she needs (within limits), and the speculator will participate in many such contracts to even out their exposure

So how do we do CFDs? In Ethereum, it's easy; just write a contract to do what you want. Here, I provide a specialized version of a CFD that I am calling a "hedging contract", which acts as a pure self-contained store of value: you put 1000 ether in, you get the same USD value of ether out (unless the value of ether drops so much that the entire contract doesn't have enough to cover you, in which case you gain the right to immediately withdraw everything and enter into a new hedging contract):

```
if contract.storage[1000] == 0: if tx.value < 1000 _ 10^18: stop contract.storage[1000] = 1
contract.storage[1001] = 998 _ block.contractstorage(D)[I] contract.storage[1002] =
block.timestamp + 30 * 86400 contract.storage[1003] = tx.sender else: ethervalue =
contract.storage[1001] / block.contractstorage(D)[I] if ethervalue >= 5000:
mktx(contract.storage[1003],5000 * 10^18,0,0) else if block.timestamp >
contract.storage[1002]: mktx(contract.storage[1003],ethervalue _ 10^18,0,0) mktx(A,(5000 -
ethervalue) _ 10^18,0,0)
```

If you understand ETH-HLL, you can figure that example out, and if you can't it basically does what the description says (the speculator puts up the contract with 4000 ETH, the counterparty enters into it with 1000 ETH, and there's an expiry date after 30 days after which anyone can "ping" the contract to return \$x worth of ETH to the counterparty and the rest to the speculator). We'll release better ETH-HLL guides soon, but for now understanding the fine details of the contract is not necessary.

trusted to create USD-backed cryptographic assets, because it requires much less infrastructure and the incentive to cheat is much smaller, but from a cryptographic purist standpoint it's not perfect. The fundamental problem is this: cryptography alone has no way of finding out that much about the outside world. You can learn a bit about computational power through proof of work, and you can get some market data between one crypto-asset and another by having an on-chain market, but ultimately there is no term in mathematical algorithms for something like the temperature in Berlin. There is no inherent way cryptography can tell you whether the correct answer is 11'C, 17'C or 2725'C; you need human judgement for that (or thermometers, but then you need human judgement to determine which thermometers are trustworthy).

Schelling time

Here, I provide a mechanism that allows you to create a decentralized data feed. The economics of it are not perfect, and if large collusions are possible then it may break down, but it is likely close to the best that we can do. In this case, we will use the price of ETH/USD as an example; the temperature in Berlin, the world GDP or even the result of a computation that does not lend itself to efficient verifiability can also be used.

The mechanism relies on a concept known as [Schelling points](#). The way it works is as follows. Suppose you and another prisoner are kept in separate rooms, and the guards give you two identical pieces of paper with a few numbers on them. If both of you choose the same number, then you will be released; otherwise, because human rights are not particularly relevant in the land of game theory, you will be thrown in solitary confinement for the rest of your lives. The numbers are as follows:

14237 59049 76241 81259 90215 100000 132156 157604

Which number do you pick? In theory, these are all arbitrary numbers, and you will pick a random one and have a probability of 1/8 of choosing the same one and getting out of prison. In practice, however, the probability is much higher, because most people choose 100000. Why 100000? Because each prisoner believes that the number 100000 is somehow "special", and each prisoner believes that the other believes that 100000 is "special", and so forth infinitely recursively – an instance of [common knowledge](#). Thus each prisoner, believing that the other is more likely to choose 100000, will choose 100000 themselves. Obviously, this is an infinitely recursive chain of logic that's not ultimately "backed" by anything except itself, but cryptocurrency users reading this article should by now be [very comfortable with relying on such concepts](#).

This mechanism is how SchellingCoin works. The basic protocol is as follows:

2. During the block after, users can submit the value whose hash they provided in the previous block.
3. Define the “correctly submitted values” as all values N where $H(N+ADDR)$ was submitted in the first block and N was submitted in the second block, both messages were signed/sent by the account with address $ADDR$ and $ADDR$ is one of the allowed participants in the system.
4. Sort the correctly submitted values (if many values are the same, have a secondary sort by $H(N+PREVHASH+ADDR)$ where $PREVHASH$ is the hash of the last block)
5. Every user who submitted a correctly submitted value between the 25th and 75th percentile gains a reward of N tokens (which we’ll call “schells”)

The protocol does not include a specific mechanism for preventing sybil attacks; it is assumed that proof of work, proof of stake or some other similar solution will be used.

So why does this work? Essentially, for the same reason why the prisoner example above worked; the truth is arguably the most powerful Schelling point out there. Everyone wants to provide the correct answer because everyone expects that everyone else will provide the correct answer and the protocol encourages everyone to provide what everyone else provides. Criminal investigators have been using SchellingCoin for centuries, putting prisoners into separate rooms and asking them all for their stories on what happened at a given event, relying on the fact that it’s easy to be consistent with many other people if you tell the truth but nearly impossible to coordinate on any specific lie.

Problems and Limits

What are the vulnerabilities? In general, collusion attacks. Most trivially, if any entity controls more than 50% of all votes, they can basically unilaterally set the median to whatever they want. On the other hand, if there are a near-infinite number of discrete non-communicating entities, then each individual entity has essentially zero impact on the result; realistically, there will be many entities giving the exact same value so there will not even be an opportunity to adjust the result slightly by voting falsely.

However, in the middle it gets hazy. If one entity controls 49% of votes, they can all pre-announce that they will vote for some false value, and others will also go with that value out of fear that everyone else will and if they don’t they will be left out. But here is the really fun part: even if one entity controls 1% of votes, if that entity pre-announces some false

value. However, the entity's value contains an incentive to vote for it, so theoretically that Schelling point is superior and everyone will go for it instead.

In practice, however, this is obviously absurd, in the same category as the famous result that in a prisoner's dilemma with a preset finite number of rounds the optimal strategy is to cheat every round; the argument is that on the last round there's no room to punish cheating, so the incentive is to cheat, on the second last round both players know that the other will cheat on the next round for that reason anyway so the incentive is to cheat, and so on recursively to the first round. In practice, people are not capable of processing arbitrary-depth recursion, and in this case in practice there is a massive coordination problem in unseating the dominant Schelling point, which only gets worse because everyone that benefits from the SchellingCoin has an incentive to attempt to censor any communication of an attempt to disrupt it. Thus, a 49% coalition will likely be able to break SchellingCoin, but a 1% coalition will not. Where is the middle ground? Perhaps only time will tell.

Another potential concern is micro-cheating. If the underlying datum is a value that frequently makes slight changes, which the price is, then if most participants in the SchellingCoin are simultaneously participants in a system that uses that SchellingCoin, they may have the incentive to slightly tweak their answers in one direction, trying to keep within the 25/75 boundary but at the same time push the median up (or down) very slightly to benefit themselves. Other users will predict the presence of micro-disruption, and will thus tweak their answers in that direction themselves to try to stay within the median. Thus, if people think that micro-cheating is possible, then micro-cheating may be possible, and if they do not think so then it will not be – a common result in Schelling point schemes.

There are two ways of dealing with the problem. First, we can try to define the value very unambiguously – eg. “the last ask price of ETH/USD on exchange XYZ at a time HH:MM:00”, so that a very large portion of answers end up exactly the same and there is no possibility to move the median at all by micro-cheating. However, this introduces centralization in the definition, so needs to be handled carefully. An alternative is to be coarse-grained, defining “the price of ETH/USD rounded to two significant digits”. Second, we can simply work hard to make the underlying system for picking users avoid biases, both by being decentralization-friendly (ie. proof-of-stake over proof-of-work) and by including users who are likely to have incentives in opposite directions.

Thus, if we combine SchellingCoin and contracts for difference, what we have is a cryptographic asset that I have previously identified as a holy grail of cryptocurrency: an asset which maintains a stable value and is simultaneously trust-free. Trust-free is of course a relative term; given the current [distribution of mining pools](#) Bitcoin's “trust-free” voting is far from completely free of any trust, but the challenge is to make the protocol as

decentralized sybil attack resistance), but every step toward the ultimate goal counts.

Mining for Schells

The interesting part about SchellingCoin is that it can be used for more than just price feeds. SchellingCoin can tell you the temperature in Berlin, the world's GDP or, most interestingly of all, the result of a computation. Some computations can be efficiently verified; for example, if I wanted a number N such that the last twelve digits of $3N$ are 737543007707, that's hard to compute, but if you submit the value then it's very easy for a contract or mining algorithm to verify it and automatically provide a reward. Other computations, however, cannot be efficiently verified, and most useful computation falls into the latter category. SchellingCoin provides a way of using the network as an actual distributed cloud computing system by copying the work among N parties instead of every computer in the network and rewarding only those who provide the most common result.

For added efficiency, a more intricate multi-step protocol can have one node do the computation and only use SchellingCoin to "spot-check" only a random 1% of the work, allowing for perhaps less than 2x cryptographic overhead. A deposit requirement and harsh penalties for providing an answer that turns out not to pass scrutiny can be used to limit fraud, and another option is to let anyone redo the work and "suggest" a verification index to the network to apply SchellingCoin on if they discover any faults.

The protocol described above is not a new idea; as I mentioned earlier, it is simply a generalization of a centuries-old criminal investigation practice, and in fact Bitcoin's mining algorithm basically is a SchellingCoin on the order of transactions. But the idea can potentially be taken much further, provided that the flaws prove to be surmountable. SchellingCoin for ETH/USD can be used to provide a decentralized dollar; SchellingCoin for computation can be used to provide distributed AWS (albeit with no privacy, but we can wait for efficient [obfuscation](#) for that).

Thanks to:

- Neal Koblitz, for suggesting the idea of using a spot checking repeated computation approach to provide a "useful proof of work"
- David Friedman, for introducing me to the concept of Schelling points in his "[positive account of property rights](#)"
- Thomas Schelling, for coming up with the concept in the first place
- An individual I talked to two months ago whose identity I unfortunately forgot for providing the idea of incorporating Schelling schemes into Ethereum

[← PREVIOUS POST](#)[NEXT POST →](#)

Stay Updated

Subscribe to get email notifications about the topics you care about.
Choose from research, events, security updates, and more.

Subscribe to Newsletter

Ethereum Foundation

Ethereum.org

ESP

Bug Bounty Program

Do-not-Track

Archive

Categories

Research & Development

Events

Organizational

Ecosystem Support Program

Ethereum.org



Funding Coordination Ecosystem Development

